

Nios[®] V/g ECC Lite Example Design
Agilex[™] 7 FPGA

Contents

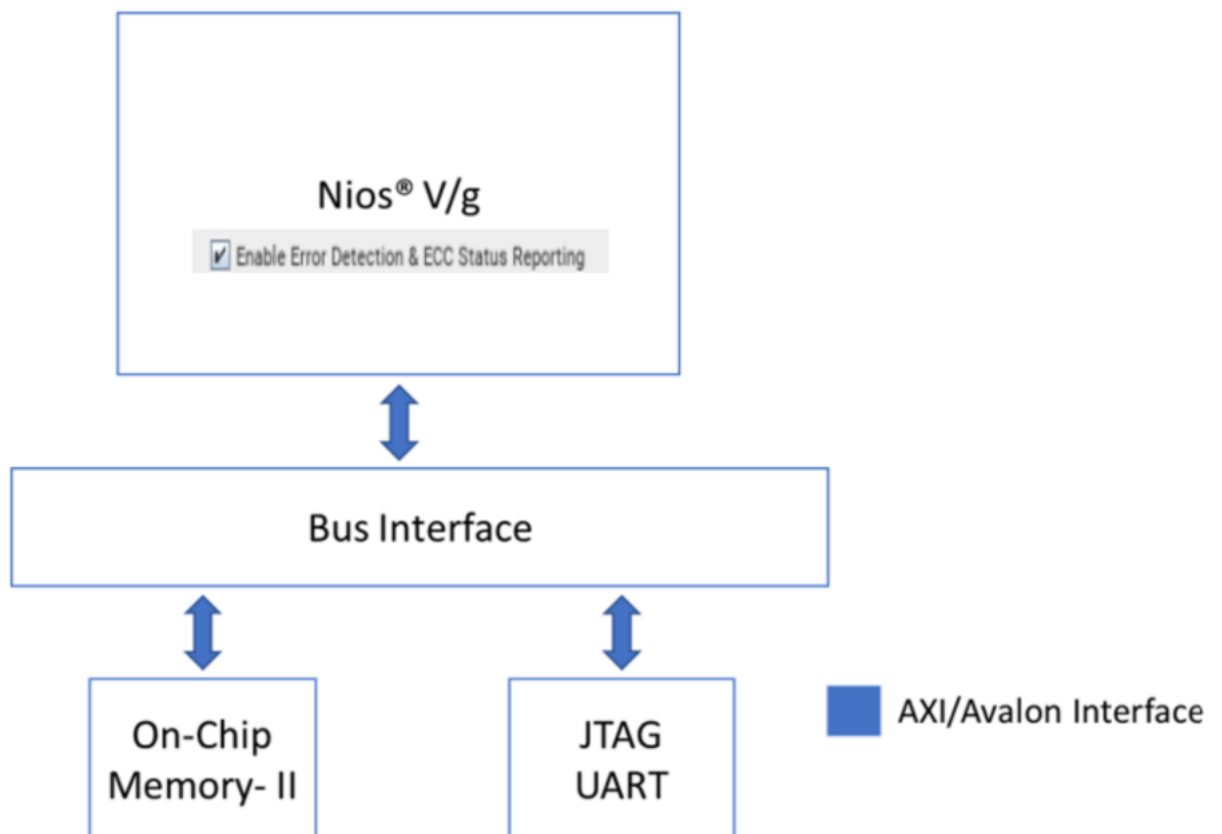
1. Theory of Operation	3
1.1. IP Cores	4
2. Directory Structure	4
3. Quick Start Guide	4
4. Generating the Design	5
4.1. Building Hardware Design	5
4.2. Building Software Design	5
5. Simulating the Design	6
6. Expected Results	7

1. Theory of Operation

This design demonstrates the ECC Lite feature of the Nios® V/g core by injecting an error on the General-Purpose Register (GPR) via simulation.

- The ECC status and ECC source is observed for both correctable and uncorrectable errors on the General-Purpose Registers (GPR).
- The Error is injected on the OCM (M20k) GPR through the ECC parity flip feature (Link).
- The parity value in the GPR is flipped using the force command in the test bench file (sys_tb.v). (Pg. 64 in Link)
- The ECC Status and ECC Source signals are probed and observed using Questa Simulation

Figure 1-1. Block Diagram



1.1. IP Cores

The following IPs are used in this design.

- Nios V/g soft processor core
- On Chip RAM-II
- JTAG UART

2. Directory Structure

The directory structure is explained below:

Root directory: agf014ea-dev-devkit/niosv_g/ecc_lite

Directories	Content
docs	Document capturing the details of the example design
img	Block diagram for the example designs
ready_to_test	Prebuilt binaries (FPGA <code>.sof</code> file and Application <code>.elf</code> file) which can be programmed on the board and tested
sources - hw - scripts - sw/app	Files needed to create the design - Necessary hardware files (<code>.sv</code>, <code>.v</code>, <code>.ip</code>) of the design - This folder consists of scripts to build the design - This folder contains software application files

3. Quick Start Guide

You may try the example on the target development kit using the prebuilt binaries. The respective FPGA `sof` and Application `elf` files are found in `ready_to_test` folder.

Refer to [Chapter 6 Programming the Design](#) for the steps to program these files.

4. Generating the Design

The implementation of Nios® V processor system requires a hardware design developed using the Quartus® Prime software. After that, it is required to develop the software design that complements the processor system.

4.1. Building Hardware Design

Begin designing the system hardware by instantiating the Nios® V processor and its peripherals into Platform Designer. After configuring the system assignments and constraints, complete the hardware design by performing a successful compilation.

We will be using `sources/scripts/build_sof.py` to develop the hardware design.

This script will create a new platform design, add the IPs into the design, makes the connections, compiles the design and generates the `.sof` file.

Steps to execute the script are as follows:

1. Invoke the Nios V Command Shell in the terminal
2. Run the following command in the terminal from top level project directory:
`> quartus_py ./scripts/build_sof.py`
3. The Quartus tool will compile the design and generate the output files into `<Root Directory>/sources/hw/output_files`

4.2. Building Software Design

After the processor system is ready, you may begin building the software design. It consists of the following steps:

1. Create a board support package (BSP) project.
2. Create a Nios® V processor application project with source code.
3. Build the application.
4. Convert the application `.elf` file into `.hex` file for memory initialization.

To create software application, run the following commands in the terminal:

```
> niosv-bsp -c --quartus-project=hw/top.qpf --qsys=hw/sys.qsys --type=hal
sw/bsp/settings.bsp

> niosv-app --bsp-dir=sw/bsp --app-dir=sw/app --srcs=sw/app/hello.c

> cmake -S ./sw/app -B sw/app/build

> make -C sw/app/build

> elf2hex sw/app/build/app.elf -b 0x0 -w 32 -e 0xffffffff ./hw/onchip_mem.hex
```

Note: It is recommended to clean the `app/build` project before regenerating `elf` (`cmake` and `make`).

5. Simulating the Design

You may carry on simulation for this design to evaluate its intended functions. During simulation, the action of downloading .elf file is unable to be simulated. Thus, the processor memory is instead initialized with the application .hex file.

Use the following commands to run the simulation:

```
# Generate Testbench from Platform Designer.

# Generate -> Generate Testbench System

> cp ./hw/onchip_mem.hex ./sys_tb/sys_tb/sim/mentor

> cd hw/sys_tb/sys_tb/sim/mentor/

> vsim &

> source msim_setup.tcl

> ld_debug

> run -all
```

6. Expected Results

The following is the output as observed on the JTAG UART terminal. The output is analogous to the logic from the application code. Users should be able to observe the same output on their terminal/setup.

Figure 1-2. Expected Result

